



Projeto 1 - Introdução à Programação

Enzo Reitano Capuano - 11370237

Prof. José Abel Hoyos Neto - *Introdução à Física Computacional (2026/2)*

A leitura através do [link](#) conta com realçador de sintaxe pros códigos

Tarefa-1: Torus

Descrição e Lógica

Estrutura

Código e Prints

Tarefa-2: Paralelepípedo de Base Triangular

Descrição e Lógica

Estrutura

Código e Prints

Tarefa-3: Ordenador

Descrição e Lógica

Programa

Subrotinas

Exemplo de Max Heap para $M = 7$

Estrutura

Código e Prints

Tarefa-4a: Série Cosseno Simples Precisão

Descrição e Lógica

Estrutura

Tarefa-1: Torus

Descrição e Lógica

A tarefa-1 é uma calculadora da área superficial total A e do volume V de um Torus, dados os raios de seu círculo menor, $r1$, e de seu círculo maior, $r2$.

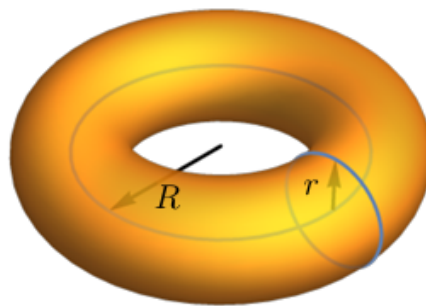


Figura 1: Um Torus. Nesta imagem $r = r1$; $R = r2$.

O usuário deve fornecer no terminal os valores de $r1$ e $r2$ e o programa os aplica nas equações:

$$r1 \cdot r2 = r, \tag{1}$$

$$A = 4\pi^2 r, \tag{2}$$

$$V = 2\pi^2 r \cdot r1, \tag{3}$$

Por fim, imprime no terminal os valores obtidos para A e V .

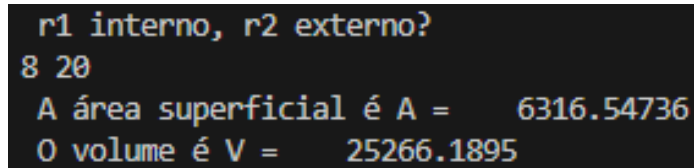
Estrutura

1. Define-se π como $4 \cdot \text{atan}(1)$;
2. Pede-se pro usuário preencher no terminal os valores de $r1$ e $r2$;

3. Calcula-se, respectivamente, as equações (2), (3) e (4);
4. Imprime-se no terminal os resultados de A e V .

Código e Prints

```
pi = 4*atan(1.e0)
write(*,*) 'r1 c.menor, r2 c.maior?'
read(*,*) r1,r2
r = r1 * r2
pi2 = pi**2
v = 2*pi2*r*r1
a = 4*pi2*r
write(*,*) 'A área superficial é A = ', a
write(*,*) 'O volume é V = ', v
end
```



```
r1 interno, r2 externo?
8 20
A área superficial é A =    6316.54736
O volume é V =    25266.1895
```

Figura 2: Exemplo Tarefa-1.

Tarefa-2: Paralelepípedo de Base Triangular

Descrição e Lógica

O usuário fornece três vetores de três componentes via terminal, $\vec{v}_1$, $\vec{v}_2$, $\vec{v}_3$.

Os vetores $\vec{v}_1$, $\vec{v}_2$ e $\vec{v}_1 - \vec{v}_2$ definem uma base triangular e $\vec{v}_3$ define a direção das arestas laterais.

O objetivo é calcular o volume total e a área lateral total do respectivo paralelepípedo formado.

Com isso, primeiro o programa confirma que os dois vetores da base não são paralelos, de fato gerando uma área. Para isso, usufrui-se do fato de que o módulo do produto vetorial entre dois vetores é justamente o valor da área do paralelogramo formado por eles e, caso seja zero, indica que são paralelos. Portanto, essa confirmação equivale à pergunta

$$|\vec{v}_1 \times \vec{v}_2| \neq 0 ? \quad (4)$$

caso a resposta seja não, o programa retorna um erro e encerra.

Em seguida, é preciso confirmar que $\vec{v}_3$ não está no plano da base e, portanto, estes três vetores de fato geram um volume. Para isso, parte-se do fato de que o produto vetorial entre dois vetores *necessariamente* é perpendicular ao plano formado por eles. Sendo assim, basta calcular o produto escalar entre o vetor resultante e $\vec{v}_3$, essencialmente calculando o volume do paralelepípedo de base quadrangular gerado por eles. Esse questionamento equivale à

$$|(\vec{v}_1 \times \vec{v}_2) \cdot \vec{v}_3| \neq 0 ? \quad (5)$$

caso não seja, o programa retorna erro e encerra.

Tendo passado por essas confirmações, o volume V é calculado através do produto misto anterior dividido por dois, já que a base em questão é triangular, e a área lateral total A_L é calculada como a soma dos módulos dos produtos vetoriais entre cada vetor da base e aquele que define a direção das arestas, ou seja:

$$V = \frac{|(\vec{v}_1 \times \vec{v}_2) \cdot \vec{v}_3|}{2}, \quad (6)$$

$$A_L = |\vec{v}_1 \times \vec{v}_3| + |\vec{v}_2 \times \vec{v}_3| + |(\vec{v}_1 - \vec{v}_2) \times \vec{v}_3|. \quad (7)$$

Por fim, os resultados são impressos no terminal.

Estrutura

| MAIN_

1. Define-se as dimensões dos vetores utilizados;
2. Pede-se pro usuário inserir as componentes de cada vetor separada por espaços;
3. Realiza-se o produto vetorial entre todos os vetores em questão;
4. Confirma-se que os vetores da base geram uma área,
 - a. Caso não, o programa encerra e retorna erro;
5. Confirma-se que os vetores da base + o que define a direção das arestas geram um volume,
 - a. Caso não, o programa encerra e retorna erro;
6. Calcula-se V ;

7. Calcula-se A_L ;
8. Imprime o resultado no terminal.

| *function abss(a)* (Módulo de um vetor)

1. Define-se a dimensão do vetor;
2. Calcula-se seu módulo.

| *function PE(a,b)* (Produto Escalar)

1. Define-se a dimensão dos dois vetores;
2. Define-se um contador;
3. Cria-se um laço $i=1,3$:
 - a. Calcula-se o produto $a(i)*b(i)$,
 - b. Acrescenta-se o produto calculado ao contador;
4. Retorna a soma final armazenada no contador.

| *subroutine PV(a,b,c)* (Produto Vetorial)

1. Define-se a dimensão dos três vetores;
2. Calcula-se as três componentes do vetor resultante, C ;
3. Retorna C .

Código e Prints

```
dimension v1(3),v2(3),v3(3),v12(3),v13(3),v23(3),v123(3)
write(*,*) 'v1 (componentes separadas por espaço)?'
read(*,*) v1
```

```

write(*,*) 'v2 (componentes separadas por espaço)?'
read(*,*) v2
write(*,*) 'v3 (componentes separadas por espaço)?'
read(*,*) v3

call PV(v1,v2,v12) !PV calcula o prod.vetorial
call PV(v1,v3,v13)
call PV(v2,v3,v23)
call PV(v1-v2,v3,v123)

if (abss(v12).eq.0) then !abss calcula o mód. do vetor
  write(*,*) 'v1 e v2 são paralelos!'
  goto 77
endif

if (pe(v12,v3).eq.0) then
  write(*,*) 'v3 esta no plano gerado por v1 e v2!'
  goto 77
endif

vol=(abs(pe(v12,v3)))/2
al= abss(v13) + abss(v23) + abss(v123)

write(*,*) 'volume=',vol
write(*,*) 'arealateraltotal=',al

```

```

77  continue

```

```

end

```

```

function abss(a) !modulo de um vetor
  dimension a(3)
  abss = sqrt(a(1)**2+a(2)**2+a(3)**2)
end

```

```

function PE(a,b) !p.escalar entre dois vetores
  dimension a(3),b(3)
  pe=0e0
  do i=1,3
    c=a(i)*b(i)

```

```

        pe=pe+c
    enddo
    return
end

subroutine PV(a,b,c) !p.vetorial entre dois vetores
    dimension a(3),b(3),c(3)

    c(1) = a(2)*b(3) - a(3)*b(2)
    c(2) = a(3)*b(1) - a(1)*b(3)
    c(3) = a(1)*b(2) - a(2)*b(1)

    return
end

```

```

v1 (componentes separadas por espaço)?
1 0 0
v2 (componentes separadas por espaço)?
0 2 0
v3 (componentes separadas por espaço)?
0 0 3
volume= 3.00000000
arealateraltotal= 15.7082043

```

Figura 3: Output Tarefa-2.

Tarefa-3: Ordenador

Descrição e Lógica

Programa

O usuário fornece um arquivo input com N números, 1/ linha, e insere pelo terminal o valor $M \leq N$. O programa deve imprimir N no terminal e criar um arquivo output onde será impresso M e os M menores números do arquivo input ordenados.

Como, nesse caso específico, o arquivo input é dado, realizei uma leitura prévia para alocar a memória exata necessária já que FORTRAN77 não possui alocação dinâmica de memória, a fim de

evitar gastos desnecessários e assim definindo a dimensão vetor $A(i)$. De forma genérica, poderia ter-se definido a dimensão do vetor na ordem de grandeza esperada pro arquivo.

O processo de ordenação aqui usado usufrui do conceito de *Heap* (exemplo na seção seguinte). Trata-se de uma estrutura de dados em forma de árvore binária que mantém uma ordem parcial entre seus elementos, podendo esta ser de dois tipos.

Definindo-se como *nó* o i -ésimo elemento $A(i)$, o *esquerdo* $A(2i)$ e o *direito* $A(2i + 1)$, temos:

- *Max Heap*: $A(i) > A(2i + 1), A(2i), \forall i$. {A raiz, $A(1)$, sempre será o *maior* valor};
- *Min Heap*: $A(i) < A(2i + 1), A(2i), \forall i$. {A raiz, será sempre o *menor* valor}.

Neste exercício em particular, foi utilizado um *Max Heap*.

Em essência, o programa começa pedindo pro usuário inserir M no terminal. Com isso, ele inicia a leitura do arquivo input, registrando cada valor lido no vetor e aumentando o contador para determinar N .

Quando se atinge $A(M)$ aplica-se a estrutura de max heap desde o primeiro elemento até ele. Portanto, a raiz $A(1)$ será sempre o maior valor destes M primeiros elementos de A .

Com isso, o programa continua a leitura do arquivo. Para cada número lido ele se pergunta se este é menor ou não do que a raiz e, caso seja, ele os troca, $A(1) \leftrightarrow A(i)$, e reaplica a estrutura desde a raiz até $A(M)$.

Portanto, ao final de apenas *uma varrida*, os primeiros M elementos de A serão necessariamente os M menores números do arquivo e estarão em estrutura de max heap! Por fim, basta ordená-los.

Para isso, substitui-se o valor na raiz pelo último, $A(1) \leftrightarrow A(M)$, e reaplica-se a estrutura abaixo dele, sem alterar a posição dos elementos recém trocados. A lógica se repete, substituindo o segundo elemento pelo penúltimo, $A(2) \leftrightarrow A(M - 1)$, sem alterar a posição do elementos recém trocados, e assim por diante.

Dessa forma, os primeiros M elementos de A , que antes eram um max heap, agora estão ordenados em ordem crescente. O programa os imprime no arquivo output.

Subrotinas

A subrotina $b_h(a, m)$ (Build Heap de tamanho m no vetor a) basicamente aplica a outra subrotina $h_d(a, i, m)$ (Heapify Down no vetor a , do i -ésimo até o m -ésimo elemento) para $i = \frac{M}{2}, 1, -1$, ou seja, começando em $M/2$ até chegar em 1 em passos de -1. Foi escolhido este valor inicial pois, para um heap de M elementos, o último nó será sempre o de índice $\frac{M}{2}$.

A subrotina $h_d(a, i, m)$ começa definindo os índices esquerdo ($il = 2i$), direito ($ir = 2i + 1$), e o teste (ix), que inicialmente ele assume como sendo o índice do nó, ($ix = i$). Assim, ele confirma

se o índice do esquerdo está no intervalo permitido ($il \leq M$). Caso esteja, ele confirma se o valor do esquerdo é maior do que o valor do teste, ($A(il) > A(ix)$). Caso seja, o índice teste torna-se o esquerdo, ($ix = il$), e repete-se o mesmo procedimento pro direito. Por fim, ele confirma se o índice teste segue sendo o do nó, ($ix = i$). Caso não seja, ele substitui os valores, $A(i) \leftrightarrow A(ix)$, e define o novo nó a partir do teste ($i = ix$) recomeçando o processo a partir deste.

Exemplo de Max Heap para $M = 7$

Suponha um vetor de 7 elementos $A = (63, 37, 33, 90, 77, 59, 21)$. Este vetor pode ser visto em estrutura de árvore binária da seguinte forma:



Figura 4: Vetor A visto em estrutura de árvore binária (heap).

Note que, conforme dito previamente, dado um elemento de índice i , define-se um nó e dois valores relacionados à ele com índices $2i$ (esquerdo) e $2i + 1$ (direito).

Agora, basta aplicar a estrutura de max heap. A raiz, $A(1)$, será o maior número.

Para isso, começa-se do último nó, $\frac{M}{2} = \frac{7}{2}$, ou seja, $A(3)$ já que deve se arredondar para baixo. Se o esquerdo (ou direito) for maior que o nó, substitui-se um pelo outro, nesse caso $A(3) \leftrightarrow A(6)$.



Figura 5: Substitui-se o esquerdo pelo nó.

Aplica-se o mesmo processo para o nó $\frac{M}{2} - 1$, nesse caso, $A(2)$.



Figura 6: Heapifica o nó $A(2)$.

Essa lógica segue até a raiz.



Figura 7: Aplica-se a estrutura para o nó $A(1)$.

Note que, ao aplicar a estrutura para a raiz, $A(2)$ agora quebra a estrutura. A subrotina h_d conta com essa possibilidade, de forma que, sempre ao aplicar a estrutura em um dado nó, ela o

acompanha até o último elemento, re-aplicando a estrutura.

Por fim, obtem-se uma árvore binária parcialmente ordenada, um max heap, de 7 elementos.



Figura 8: Max Heap de 7 elementos.

Estrutura

| *MAIN__*

1. Define-se:
 - a. As variáveis reais de dupla precisão $A(50001)$ e b ;
 - b. O contador de números do arquivo input $n = 1$;
2. São abertas as unidades e os respectivos arquivos de input e output;
3. Pede-se para o usuário inserir M no terminal e escreve este valor no arquivo output;
4. Inicia-se a leitura do arquivo input, atribuindo o valor lido à $A(n)$,
 - a. Quando $(n = M)$, chama-se $b_h(a,m)$,
 - b. Quando $(n > M)$, compara-se $A(n)$ com $A(1)$,
 - i. Se $(A(n) < A(1))$, substitui-se $A(n) \leftrightarrow A(1)$ e chama-se $h_d(a,1,m)$;
 - c. Calcula-se $n = n + 1$ e volta à linha de leitura;
5. Escreve-se $N = n - 1$ no terminal;
6. Ordena-se o max heap resultante:

- a. Define-se $k = M, 2, -1$,
 - i. Substitui-se $A(1) \leftrightarrow A(k)$,
 - ii. Chama-se $h_d(a, 1, k-1)$;
7. Imprime-se os M primeiro valores de A no arquivo output;
8. Fecha-se as unidades de input e output.

| $b_h(a, m)$

1. Define-se o real de dupla precisão $A(*)$;
2. Cria-se um laço que começa do último nó e vai até a raiz nó por nó, ou seja, $i = m/2, 1, -1$,
 - a. Heapifica nó por nó com $h_d(a, i, m)$;
3. Retorna o vetor A heapificado até o m -ésimo elemento.

| $h_d(a, j, m)$

1. Define-se as variáveis reais de dupla precisão $A(*)$, b , e $i = j$;
2. Cria-se um laço onde começa definindo-se os índices relacionados ao nó i em que será aplicado a estrutura max heap:
 - a. $il = 2i$,
 - b. $ir = 2i + 1$,
 - c. $ix = i$;
3. Caso exista $A(il)$, $(il < m)$?
 - a. Se $(A(il) > A(ix))$, substitui-se os índices $il \leftrightarrow ix$;
4. Caso exista $A(ir)$, $(ir < m)$?
 - a. Se $(A(ir) > A(ix))$, substitui-se os índices $ir \leftrightarrow ix$;
5. Se $(ix = i)$
 - a. Finaliza a subrotina e retorna;

6. Substitui-se os elementos $A(i) \leftrightarrow A(ix)$, e os índices $i = ix$;
 7. O loop repete até que $(ix = i)$;
 8. A subrotina retorna o vetor a devidamente heapificado com *max heap* a partir do i -ésimo nó até o m -ésimo elemento.
-

Código e Prints

```
real*8 a(500001), b
n=1

open(unit=10,file='tarefa-3-entrada-1.in')
open(unit=11,file='tarefa-3-saida-1.out')

write(*,*) 'M = ?'
read(*,*) m
write(11,*) 'M = ', m

10  read(10,*,end=20) a(n)
      if (n.eq.m) then !aplica-se a estrutura ao atingir m
          call b_h(a,m)
      endif
      if (n.gt.m) then
          if (a(n).lt.a(1)) then !confirma se o elemento lid
o é menor que a raiz
              b = a(1)
              a(1) = a(n)
              a(n) = b
              call h_d(a,1,m) !reorganiza a estrutura
          endif
      endif
      n=n+1
      goto 10

20  write(*,*) 'N = ', n-1

      do k=m,2,-1 !ordena o heap em ordem crescente
          b = a(1)
          a(1) = a(k)
```

```

        a(k) = b
        call h_d(a,1,k-1)
    enddo

    do i=1,m
        write(11,*) a(i)
    enddo

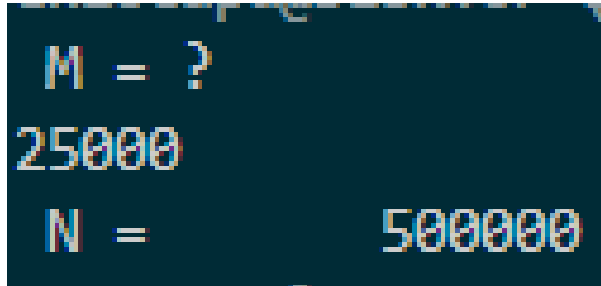
    close(10)
    close(11)
end

subroutine b_h(a,m)
    real*8 a(*)
    do i=m/2,1,-1 !constrói o heap do último nó até a raiz
        call h_d(a,i,m)
    enddo
    return
end

subroutine h_d(a,j,m)
    real*8 a(*), b
    i=j
50    continue
    il=2*i !índice esquerdo
    ir=2*i+1 !índice direito
    ix=i !índice do maior (inicialmente, o nó)
    if (il.le.m) then
        if (a(il).gt.a(ix)) ix=il !esquerdo>maior?
    endif
    if(ir.le.m) then
        if (a(ir).gt.a(ix)) ix=ir !direito>maior?
    endif
    if(ix.eq.i) then !maior = nó?
        goto 40
    endif
    b = a(i)
    a(i) = a(ix)
    a(ix) = b
    i=ix

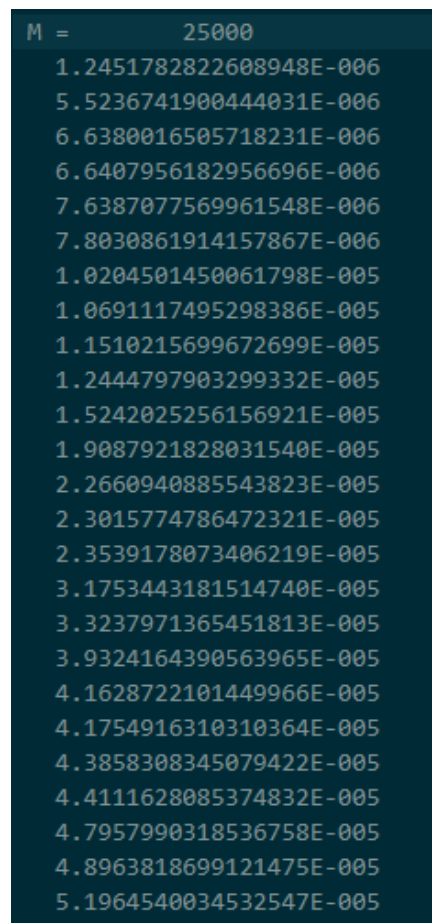
```

```
goto 50
40    return
end
```



A terminal window with a dark background. The text 'M = ?' is displayed in a large, stylized font. Below it, '25000' is shown. Further down, 'N = 500000' is displayed in the same large, stylized font.

Figura 8: Terminal tarefa-3.



A screenshot of a text file. The first line is 'M = 25000'. Below it is a list of 20 numbers in scientific notation, each on a new line. The numbers are: 1.2451782822608948E-006, 5.5236741900444031E-006, 6.6380016505718231E-006, 6.6407956182956696E-006, 7.6387077569961548E-006, 7.8030861914157867E-006, 1.0204501450061798E-005, 1.0691117495298386E-005, 1.1510215699672699E-005, 1.2444797903299332E-005, 1.5242025256156921E-005, 1.9087921828031540E-005, 2.2660940885543823E-005, 2.3015774786472321E-005, 2.3539178073406219E-005, 3.1753443181514740E-005, 3.3237971365451813E-005, 3.9324164390563965E-005, 4.1628722101449966E-005, 4.1754916310310364E-005, 4.3858308345079422E-005, 4.4111628085374832E-005, 4.7957990318536758E-005, 4.8963818699121475E-005, 5.1964540034532547E-005.

Figura 9: Arquivo output tarefa-3.

Tarefa-4a: Série Cosseno Simples Precisão

Descrição e Lógica

A tarefa-4a busca uma comparação direta entre a função intrínseca $\cos(x)$ do *FORTTRAN77* com uma implementação manual da expansão de Taylor em torno de $x_0 = 0$ da função, isto é

$$\cos(x) = \sum_{n=0}^{\infty} (-1)^n \frac{x^{2n}}{(2n)!}. \quad (8)$$

O usuário insere o valor de $x \in \mathbb{R}$ via terminal, com precisão *REAL*4*, e o resultado do somatório deve ter precisão $\epsilon = 10^{-5}$ em comparação à função intrínseca.

A precisão do somatório é definida a partir de seu *resto* R_N , definido como

$$R_N = S - S_N, \quad (9)$$

onde S representa o somatório (8) e S_N representa o mesmo somatório porém apenas até o N -ésimo termo. A partir dessa definição, conclui-se o seguinte

$$\begin{aligned} R_N &= \sum_{N+1}^{\infty} (-1)^n a_n, \\ &= (-1)^{N+1} (a_{N+1} - a_{N+2} + \dots) \end{aligned} \quad (10)$$

onde $a_n = x^{2n}/(2n)!$.

Além disso, o teste da razão indica que

$$\lim_{n \rightarrow \infty} \frac{a_{n+1}}{a_n} = \lim_{n \rightarrow \infty} \frac{x^2}{(2n+2)(2n+1)} = 0, \quad (11)$$

logo, não só a série converge como, aliado à equação (10), podemos concluir

$$\begin{aligned} |R_N| &= |a_{N+1} - a_{N+2} + \dots| \\ &= |a_{N+1} - (a_{N+2} - a_{N+3} + \dots)| \\ &\leq |a_{N+1}|, \end{aligned} \quad (12)$$

definindo a precisão do somatório.

O programa também aproveita do fato de que a função cosseno é periódica, $\cos(x) = \cos(x + 2k\pi)$, $\forall k \in \mathbb{Z}$, dessa forma reduzindo o x inserido para o intervalo $[-\pi, \pi]$ através da função *Módulo* que retorna o resto inteiro de uma divisão. Isso é útil pois, para variáveis *REAL*4*, valores grandes de x introduzem imprecisão ao realizar repetidas operações aritméticas.

Estrutura

| *MAIN_*

1. Define-se
 - a. π ,
 - b. O sinal alternante;
2. Começa-se:
 - a. O somatório,
 - b. O fatorial,
3. Pede-se pro usuário inserir x via terminal e registra-se o valor;
4. Calcula-se $\cos(x)$ via função intrínseca;
5. Se reduz x via resto;
6. Inicia-se um laço:
 - a. Calcula-se:
 - i. O fatorial deste termo,
 - ii. O termo dividido pelo fatorial multiplicado pelo sinal;
 - b. Compara-se o termo atual com o resto do somatório:
 - i. Caso seja menor que a precisão almejada, $\epsilon = 0.5e - 6$, sai do laço;
 - c. Soma-se o termo atual ao somatório;
 - d. Multiplica o sinal por -1 ;
7. Imprime no terminal o valor final do somatório, o valor via função intrínseca e a diferença absoluta;
8. Define-se a formatação com 5 casas decimais.

Código e Prints

```
pi=4.0*atan(1.0)
s=1 !definindo a série
```

```

j=1 !definindo o fatorial
isinal=-1
write(*,*) 'x= ?'
read(*,*) x
c=cos(x)

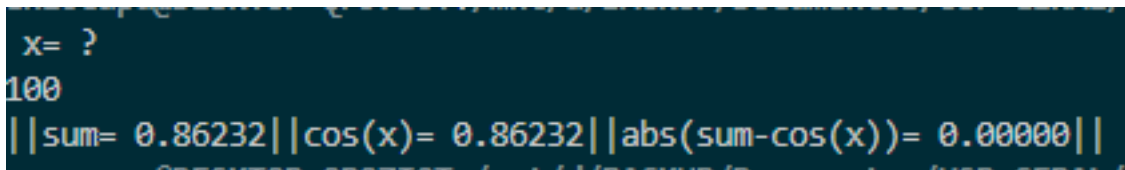
x=amod(x,2.0*pi) !calcula-se o resto
if (x.gt.pi) x=x-2.0*pi !traz x pro intervalo [-\pi,\pi]
if (x.lt.-pi) x=x+2.0*pi

do i=2,40,2
    j=i*(i-1)*j
    b=isinal*(x**i)/j
    if (abs(b).le.0.5e-5) goto 77
    s=s+b
    isinal=isinal*(-1)
enddo

77  write(*,10) '||sum=',s,'cos(x)=',c,'abs(sum-cos(x))=',abs(s-c)
10  format(3(A,F8.5,'||'))

end

```



```

x= ?
100
||sum= 0.86232||cos(x)= 0.86232||abs(sum-cos(x))= 0.00000||

```

Figura 10: Terminal tarefa-4a.

Tarefa-4b: Somatório Cosseno Dupla Precisão

Descrição e Lógica

Segue a mesma lógica da tarefa-4a, porém em dupla precisão.

O objetivo agora é testar qual o valor de ϵ necessário para que se atinja a mesma precisão da função intrínseca $\text{dcos}(x)$ em com a expansão de Taylor. Lembrando que consideram-se imprecisões de último dígito devido à ponto flutuante.

Estrutura

1. Define-se as variáveis de dupla precisão
 2. ... (segue como o anterior)
-

Código e Prints

```
real*8 pi,s,c,x,b,j
integer i,isinal

pi=4.d0*datan(1.d0)
s=1.d0
j=1.d0
isinal=-1

write(*,*) 'x= ?'
read(*,*) x

c=dcos(x)

x=dmod(x,2.d0*pi)
if (x.gt.pi) x=x-2.d0*pi
if (x.lt.-pi) x=x+2.d0*pi

do i=2,40,2
    j=i*(i-1)*j
    b=isinal*(x**i)/j
    if (dabs(b).le.0.5d-15) goto 77
    s=s+b
    isinal=-isinal
enddo

77 write(*,10) ' || sum=',s,' cos(x)=',c,' abs(sum-dcos(x))=',dabs(s-
c)

10 format(3(A,F20.15,' || '))

end
```

```
x= ?  
100  
||sum= 0.862318872287686||cos(x)= 0.862318872287684||abs(sum-dcos(x))= 0.000000000000002||
```

Figura 11: Terminal tarefa-4b.